



AWS Step Functions

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to Step Functions

What is a state machine?

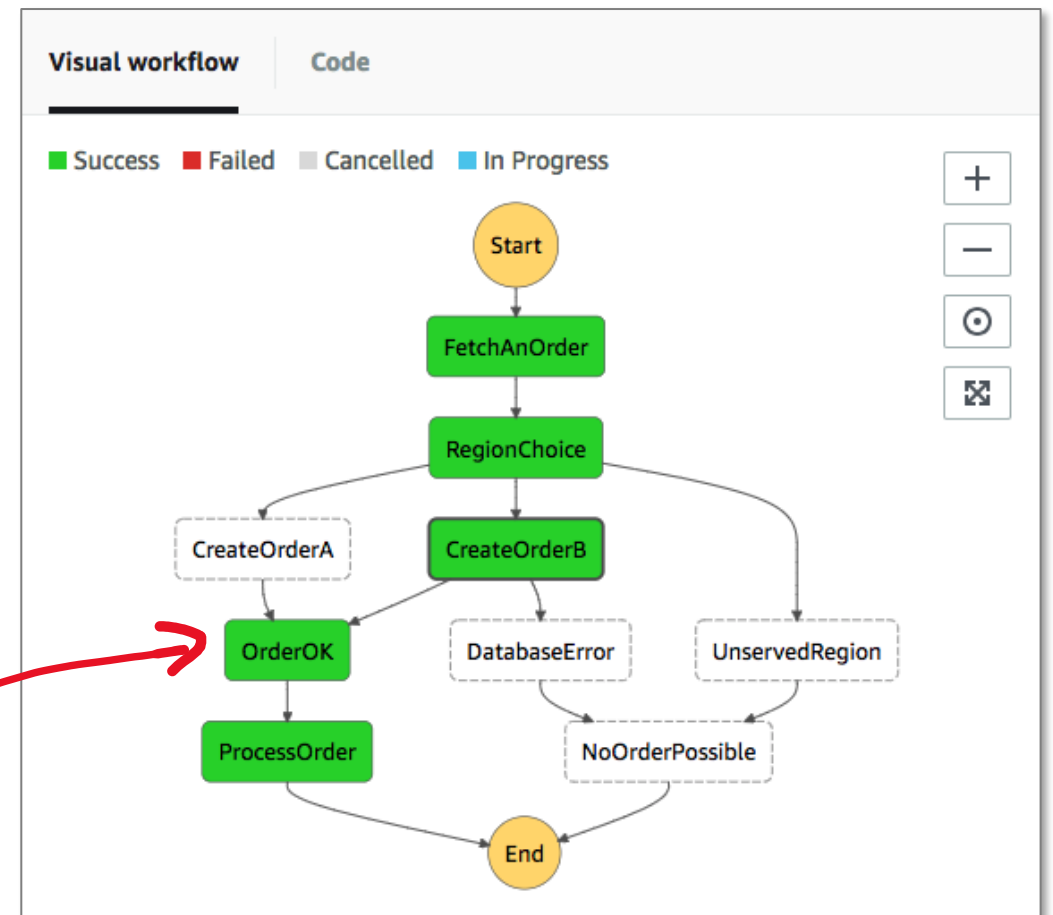
A state machine is an abstract model which decides how one state moves to another based on a series of conditions. **Think of a state machine like a flow chart.**



What is a Step Function?

- Coordinate multiple AWS Services into a serverless workflow
- A graphical console to visualize the components of your application as a series of steps.
- Automatically triggers and tracks each step and retries when there are errors, so your application executes in order and as expected every time
- logs the state of each step, so when things go wrong, you can diagnose and debug problems quickly

Any one of these steps could be using an AWS Service



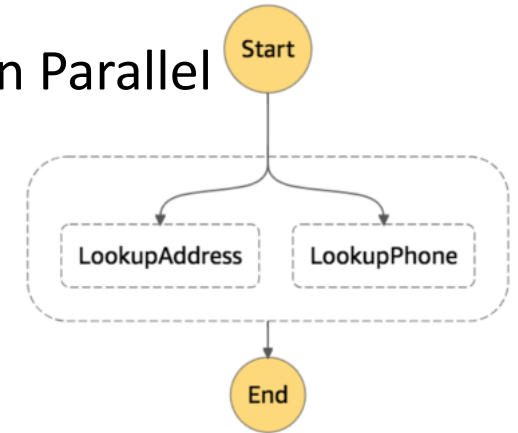
Introduction to Step Functions

Step Functions has two types of State Machines:

Standard – general purpose

Express – for streaming data

You can Execute Steps in Parallel



States are configured through
Amazon States Language (JSON)

Type
☒ Standard Durable, checkpointed workflows for machine learning, order fulfillment, IT/DevOps automation, ETL jobs, and other long-duration workloads.
☐ Express New Event-driven workflows for streaming data processing, microservices orchestration, IoT data ingestion, mobile backends, and other short duration, high-event-rate workloads.

Long workloads

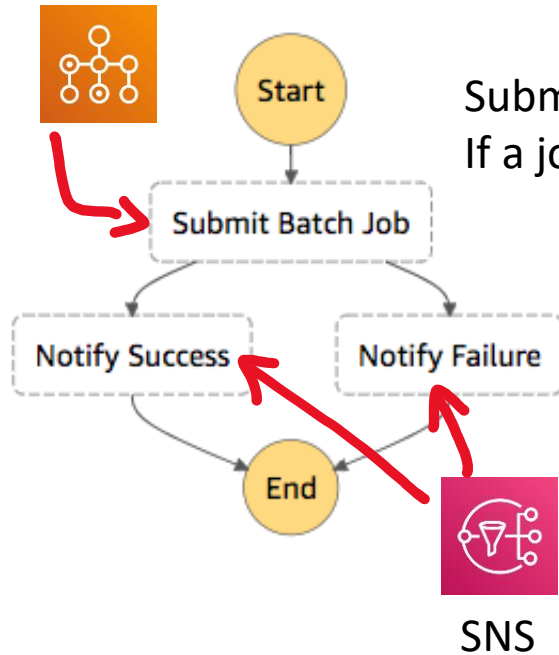
Short workloads

```
"States": {
  "Submit Batch Job": {
    "Type": "Task",
    "Resource": "arn:aws:states:::batch:submitJob.sync",
    "Parameters": {
      "JobName": "BatchJobNotification",
      "JobQueue": "arn:aws:batch:us-east-1:123456789012:j",
      "JobDefinition": "arn:aws:batch:us-east-1:123456789
    },
    "Next": "Notify Success",
    "Catch": [
      {
        "ErrorEquals": [ "States.ALL" ],
        "Next": "Notify Failure"
      }
    ]
  },
}
```

Step Functions – Use Cases

Manage a Batch Job or Fargate Container

AWS Batch

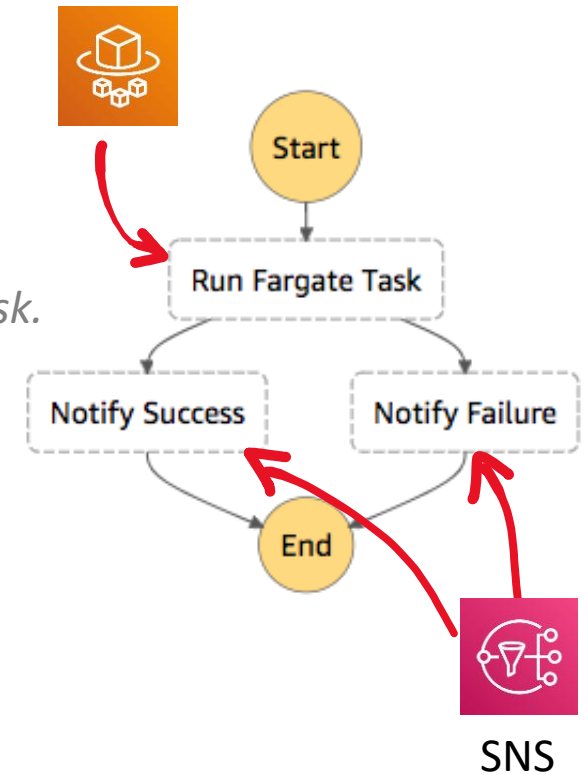


Run a **Fargate Task**

If a job fails or succeeds, notify via SNS

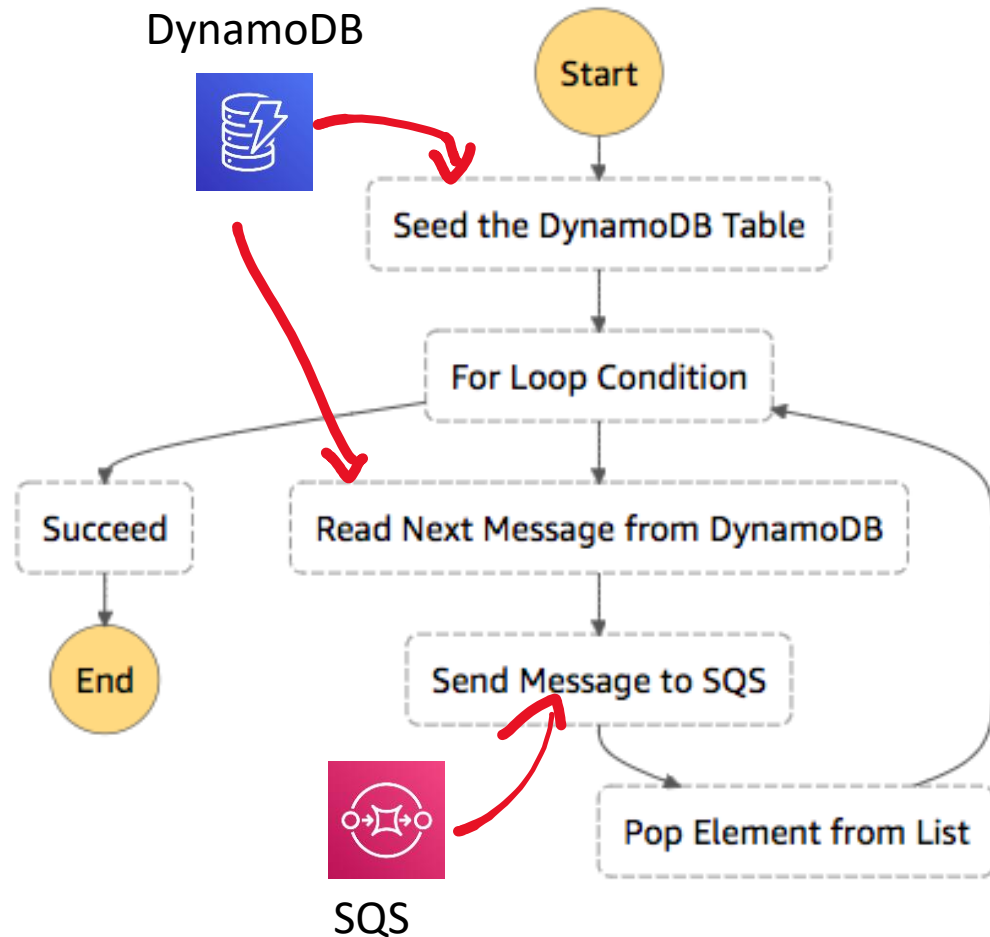
We can do the same here for ECS task.

Fargate



Step Functions – Use Cases

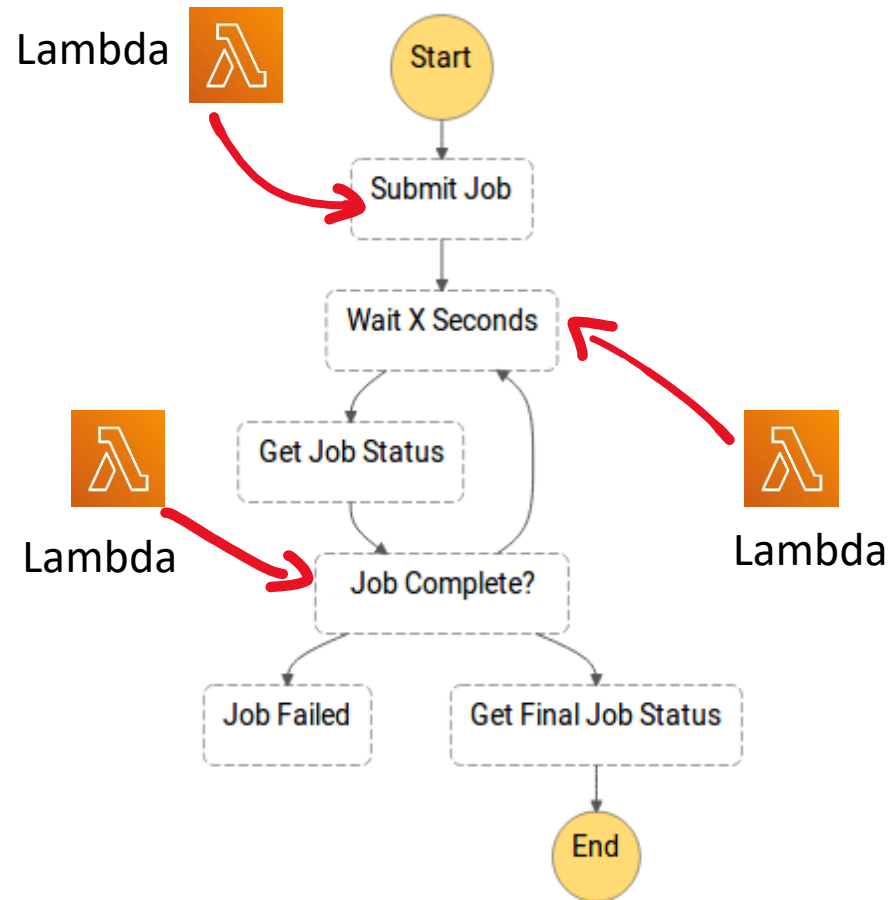
Transfer Data Records



- Load up a DyanmoDB table.
- Add each DynamoDB item to the queue
- As each item is processed, remove the item from the DynamoDB table
- When there are no more records, the report is successfully complete

Step Functions – Use Cases

Monitor batch jobs for completion



- A Lambda to submit a job to AWS Batch
- A Lambda to wait
- A Lambda to check if a job is done

Step Functions – States

Pass State

Passes its input to its output **without performing work (dummy/mock)**.

Pass states are useful when constructing and debugging state machines.

Parameters

Key-value pairs that will be passed as input

Result

A virtual task to be passed to the next state

ResultPath

Where to place the "output" of the virtual task

```
{
  "Type": "Pass",
  "Parameters": {
    "ship": "enterprise"
  },
  "Result": {
    "government": "federation"
  },
  "ResultPath": "$.politics",
  "Next": "End"
}
```

output

```
{
  "ship": "enterprise",
  "politics": {
    "government": "federation"
  }
}
```

Step Functions – States

Task State

represents a single unit of work performed by a state machine.

A task **performs work** by:

1. a **AWS Lambda** function,
2. or passing parameters to the **API actions of other services**.
3. or using an **activity**

Step Functions – States

Task – Lambda Function

You provide the Lambda ARN as the Resource



```
"LambdaState": {  
  "Type": "Task",  
  "Resource": "arn:aws:lambda:us-east-1:123456789012:function:cropPhoto",  
  "Next": "NextState"  
}
```

Step Functions – States

Task - A Supported AWS Service

You pass the ARN as the Resource

Parameters vary per service

Supported Services

- Lambda
- AWS Batch
- DynamoDB
- ECS/Fargate
- SNS
- SQS
- SageMaker
- EMR
- StepFunctions

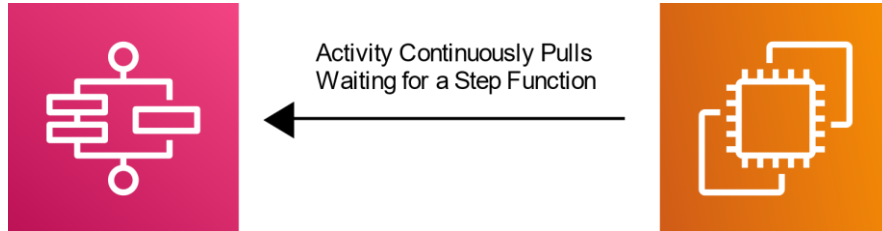
```
{
  "StartAt": "BATCH_JOB",
  "States": {
    "BATCH_JOB": {
      "Type": "Task",
      "Resource": "arn:aws:states:::batch:submitJob.sync",
      "Parameters": {
        "JobDefinition": "preprocessing",
        "JobName": "PreprocessingBatchJob",
        "JobQueue": "SecondaryQueue",
        "Parameters.$": "$.batchjob.parameters",
        "RetryStrategy": {
          "attempts": 5
        }
      },
      "End": true
    }
  }
}
```

Step Functions – States

Task – Activities

Enables you to have a task in your state machine where the work is performed by a **worker** that **can be hosted anywhere** (eg. EC2, ECS, mobile phones)

When Step Functions reaches an activity task state, the workflow waits for an activity worker to poll for a task.



```
activity = StepFunctions::Activity.new({  
  credentials: credentials,  
  region: region,  
  activity_arn: activity_arn,  
  workers_count: 1,  
  pollers_count: 1,  
  heartbeat_delay: 30  
})  
  
activity.start do |input|  
  # do something....  
  { result: :SUCCESS, echo: input['value'] }  
end
```

Step Functions – States

Choice State

Adds branching logic to a state machine.



```
"AlienChooser": {  
  "Type": "Choice",  
  "Choices": [  
    { "Not": { "Variable": "$.skin", "StringEquals": "pink" }, "Next": "Andorians"},  
    { "Variable": "$.hearts", "NumericEquals": 2, "Next": "Kligons"},  
    { "And": [  
      { "Variable": "$.legs", "NumericGreaterThanEquals": 2 },  
      { "Variable": "$.mouths", "NumericLessThan": 1 }  
    ], "Next": "Tholians"  
  ],  
  "Default": "Vulcans"  
}
```



Step Functions – States

Wait State

Delays the state machine from continuing for a specified time.

```
"wait_ten_seconds": {  
  "Type": "Wait",  
  "Seconds": 10,  
  "Next": "NextState"  
}
```

```
"wait_until" : {  
  "Type": "Wait",  
  "Timestamp": "2016-03-14T01:59:00Z",  
  "Next": "NextState"  
}
```

You could provide a variable

```
"TimestampPath": "$.expirydate"
```

Step Functions – States

Succeed State

Stops an execution successfully

```
"SuccessState": {  
  "Type": "Succeed"  
}
```

The Succeed state is a useful target for Choice state branches that don't do anything but stop the execution.

Step Functions – States

Fail State

Stops the execution of the state machine and marks it as a failure

```
"FailState": {  
  "Type": "Fail",  
  "Cause": "Overloading.",  
  "Error": "WarpCore"  
}
```

Because Fail states always exit the state machine, they have no Next field and don't require an End field.

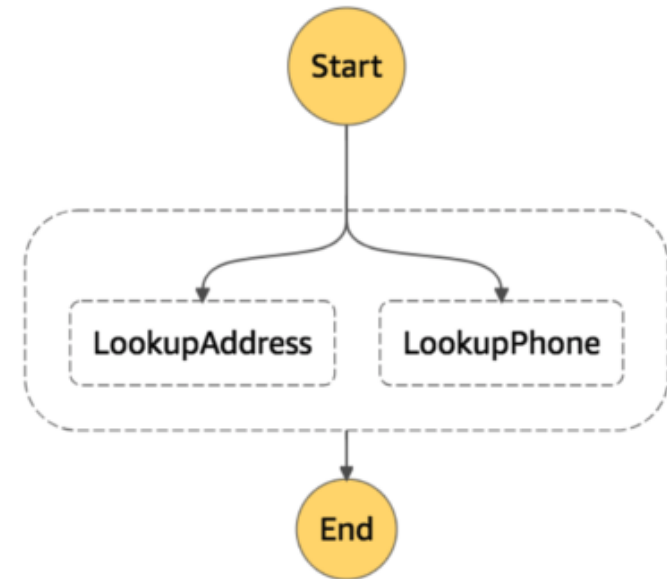
Step Functions – States

Parallel States

Can be used to create parallel branches of execution in your state machine

The state machine does not move forward until both states complete

```
{
  "Comment": "Parallel Example.",
  "StartAt": "LookupCustomerInfo",
  "States": {
    "LookupCustomerInfo": {
      "Type": "Parallel",
      "End": true,
      "Branches": [
        { "StartAt": "LookupAddress",
          "States": {
            "LookupAddress": { "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:AddressFinder", "End": true }
          }
        },
        { "StartAt": "LookupPhone",
          "States": {
            "LookupPhone": { "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:PhoneFinder", "End": true }
          }
        }
      ]
    }
  }
}
```



Step Functions – States

Map State

can be used to run a set of steps for each element of an input array.

Step Functions – Inputs and Outputs

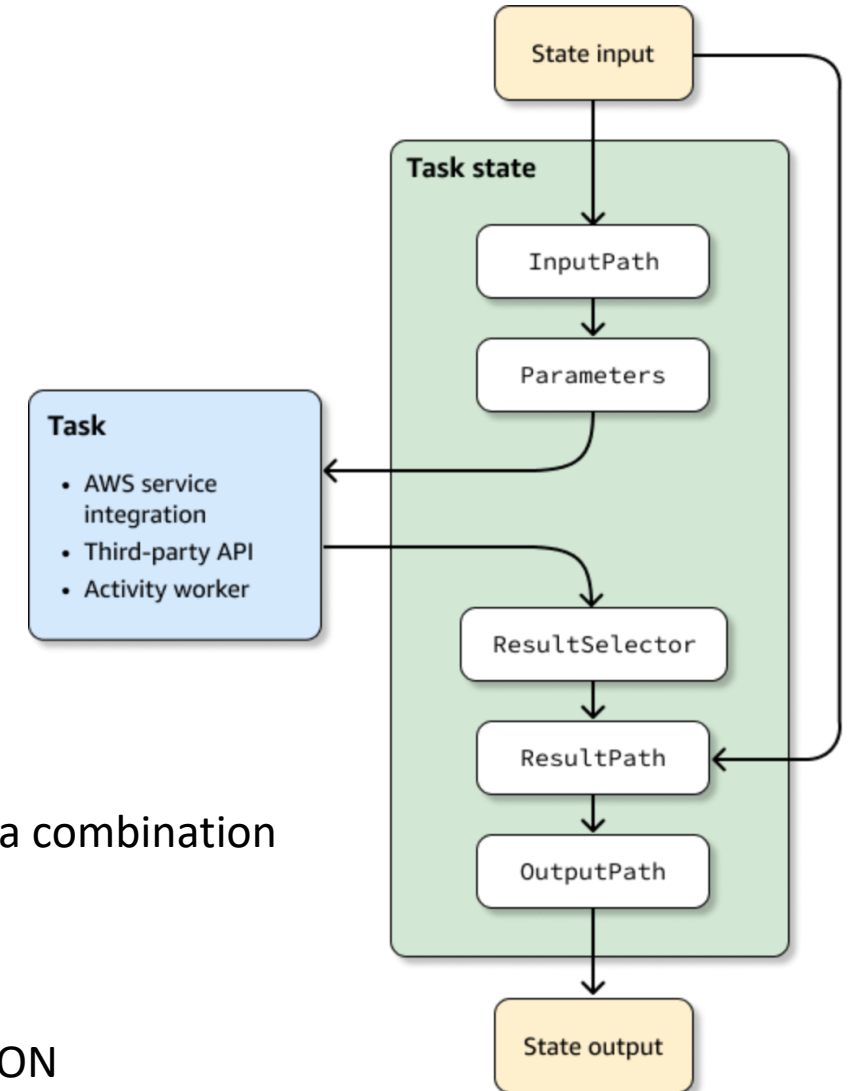
```
▼ 2 PassStateEntered Parse S3 Event
1 {
2   "input": {
3     "version": "0",
4     "id": "6fcd98ef-628f-a9e4-4d7f-778217e574",
5     "detail-type": "Object Created",
6     "source": "aws.s3",
7     "account": "982383527471",
8     "time": "2024-05-03T16:58:58Z",
9     "region": "us-east-1",
```

Step Functions will **receive JSON event data** as **input** and pass JSON as **output**.

You can manipulate this JSON payload using the following:

- InputPath — select a portion of the state input
- Parameters — create a collection of key-value pairs that are passed as input.
- ResultSelector — manipulate a state's result before ResultPath is applied
- ResultPath — determines what should be outputted, the input, task output or a combination
- OutputPath — select a portion of the state output to pass to the next state.

Amazon States Language uses **JSONPath syntax** to identify components of JSON



Step Functions – Inputs and Outputs

JSONPath is a query language for JSON, similar to XPath for XML

```
{
  "input": {
    "version": "0",
    "id": "6fcd98ef-628f-a9e4-4d7f-778217e5744a",
    "detail-type": "Object Created",
    "source": "aws.s3",
    "account": "982383527471",
    "time": "2024-05-03T16:58:58Z",
    "region": "us-east-1",
    "resources": [
      "arn:aws:s3:::sf-star-trek-131241"
    ],
    "detail": {
      "version": "0",
      "bucket": {
        "name": "sf-star-trek-131241"
      },
      "object": {
        "key": "inputs/picard.jpg",
        "size": 4879,
        "etag": "98b35d1022f5051b4ec2d8520309d1ff",
        "sequencer": "00663517D2E67AB2A0"
      }
    },
    // ...
  }
}
```

- Expression begin with a \$ which represent the **root object**
- Children can be selected using dot notation
- Children can be selected using bracket notation

```
$.input.detail.bucket.name // dot notation
$['input']['detail']['object']['key'] // bracket notation
```

JSONPath evaluator can be used to help test your jsonpath syntax against a json file

<https://jsonpath.com/>



I made a mistake in the JSONPATH syntax for inputs in these slides You just need to write \$.detail and not \$.input.detail

Step Functions – Inputs and Outputs

You can select elements at any depth



You can use wildcard to get all elements



You can apply complex filters.

@ - means the current element



The field Filter is not supported by Step Functions.

```
// The root object
$

// Dot Notation and Square Notation
$.input.detail['object']['key']

// All 'object' elements at any depth
$..object

// All children of the 'input' object
$.input.*

// First element of 'resources' array
$..resources[0]

// All elements of 'resources' array
$..resources[*]

// All elements where 'size' is greater than 1000
$..[?(@.size > 1000)]

// All elements that have both 'etag' and 'size' properties
$..[?(@.etag && @.size)]
```

A **reference path** is a path whose syntax is limited in such a way that it can **identify only a single node in a JSON structure**

Step Functions – Inputs and Outputs

InputPath allows us to select what we plan to pass to our current step

```
"States": {  
  "CurrentStep": {  
    "Type": "Pass",  
    "InputPath": "$.input.detail.bucket",  
    "Next": "NextStep"  
  },  
}
```



```
{  
  "input": {  
    "version": "0",  
    "id": "6fcd98ef-628f-a9e4-4d7f-778217e5744a",  
    "detail-type": "Object Created",  
    "source": "aws.s3",  
    "account": "982383527471",  
    "time": "2024-05-03T16:58:58Z",  
    "region": "us-east-1",  
    "resources": [  
      "arn:aws:s3:::sf-star-trek-131241"  
    ],  
    "detail": {  
      "version": "0",  
      "bucket": {  
        "name": "sf-star-trek-131241"  
      },  
      "object": {  
        "key": "inputs/picard.jpg",  
        "size": 4879,  
        "etag": "98b35d1022f5051b4ec2d8520309d1ff",  
        "sequencer": "00663517D2E67AB2A0"  
      },  
    },  
  },  
  // ...  
}
```

```
{ "name": "sf-star-trek-131241" }
```

Step Functions – Inputs and Outputs

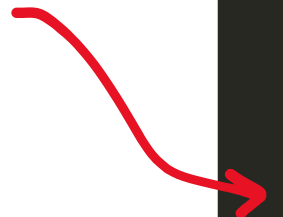
Parameters allows to construct key value pairs.
When you want to use JSONPATH for parameters
you need to add **“.\$”** to the key name

```
"States": {  
  "CurrentStep": {  
    "Type": "Pass",  
    "Parameters": {  
      "bucket.$": "$.input.detail.bucket.name",  
      "object_key.$": "$.input.detail.object.key",  
      "author": "Andrew"  
    }  
  },  
  "Next": "NextStep"  
},
```



Tasks that call AWS resources may
expect specific Parameters.

```
"LambdaState": {  
  "Type": "Task",  
  "Resource": "arn:aws:states:::lambda:invoke",  
  "OutputPath": "$.Payload",  
  "Parameters": {  
    "Payload.$": "$",  
    "FunctionName": "arn:aws:lambda:us-east-1:function:HelloWorld:$LATEST"  
  },  
  "Next": "NextState"  
}
```



Step Functions – Inputs and Outputs

ResultsSelector lets you create a collection of key value pairs, where the values are static or selected from the state's result.
(Only works for Map, Parallel or Task State)

```
{
  "Comment": "A Step Function state machine that processes data from an S3 event using EC2.",
  "StartAt": "Process Data",
  "States": {
    "Process Data": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account-id:function:function-name",
      "Parameters": {
        "bucket.$": "$.detail.bucket.name",
        "key.$": "$.detail.object.key"
      },
      "ResultPath": "$.processedData",
      "Next": "Run EC2 Task"
    },
    "Run EC2 Task": {
      "Type": "Task",
      "Resource": "arn:aws:states:::elasticmapreduce:addStep.sync",
      "Parameters": {
        "ClusterId": "YourClusterID",
        "Step": {
          "Name": "Process S3 Data on EC2",
          "ActionOnFailure": "CONTINUE",
          "HadoopJarStep": {
            "Properties": [],
            "Jar": "command-runner.jar",
            "Args.$": "States.Array('aws', 's3', 'cp', $.processedData.S3Uri, '/mnt/tmp')"
          }
        }
      }
    },
    "ResultSelector": {
      "InstanceId.$": "$.processedData.InstanceId",
      "OutputLocation.$": "$.processedData.S3Uri"
    },
    "End": true
  }
}
```

Step Functions – Inputs and Outputs

ResultPath lets you decide to:

- Use only the output from a task
- Use the input as the output
- Use the output and add or have it replace an existing key in the input and have that as the output

(Only works with Pass, Task state, Parallel, Map)

```
// the output will be whatever is return for the task  
"ResultPath": "$"
```

```
// the output will be the input  
"ResultPath": "null"
```

```
// The output from the task will replace a key mydata or create a new key mydata on the input data  
"ResultPath": "$.mydata"
```

OutputPath enables you to select a portion of the state output to pass to the next state.



Step Functions Cheat Sheet

- **Step Functions** coordinate multiple AWS services into serverless workflows using **State Machines**.
- A state machine is an abstract model which decides how one state moves to another based on a series of conditions.
Think of a state machine like a flow chart.
- Step Functions automatically trigger and track each step and retries when there are errors, so your application executes in order and as expected every time
- **Amazon States Language** is how you define all your states and is written in **JSON**.
- There are two types of State Machines available:
 - **Standard** – general purpose, long workloads
 - **Express** – streaming data, short workloads
- Use Cases for Step Functions:
 - Manage a batch job, Fargate, or ECS container
 - Sequential batch processing
 - Creating a pipeline to transcode media files
 - Transferring data records by performing operations on each record queued up via SQS
 - Sequence steps of ML workflows
 - Coordinate Extract, Transform, and Load (ETL) Jobs



Step Functions Cheat Sheet

- **The type of Step Functions States**

- **Pass State** Passes its input to its output without performing work (dummy/mock). Pass states are useful when constructing and debugging state machines.
- **Task State** represents a single unit of work performed by a state machine.
 - **Lambda** – triggers a lambda function
 - **Activity** - the work is performed by a worker that can be hosted anywhere (eg. EC2, ECS, mobile phones)
- **Supported AWS Services** pass parameters to the API of an AWS Service
- **Choices State** adds branching logic to a state machine.
- **Wait State** delays the state machine from continuing for a specified time.
- **Succeed State** stops an execution successfully
- **Fail State** stops the execution of the state machine and marks it as a failure
- **Parallel State** can be used to create parallel branches of execution in your state machine. The state machine does not move forward until both states complete
- **Map State** can be used to run a set of steps for each element of an input array.